chris-code

An opinionated Claude Code workflow

Design before it codes. Review before it lands.

`25 skills · 13 agents · uniform review gates`

What it is

A personal **Claude Code plugin** that turns the assistant from a free-form chat tool into a **fixed engineering pipeline**.

Drop it into any repo. Describe a feature (or type `/brainstorming`) and the workflow takes over:

> brainstorm intent → write a lean spec → hand off a thin plan → dispatch a coder per task → review in stages → gate every commit.

Derived from obra/superpowers, redesigned around three failures it kept hitting:

Why it exists: three failures

1

Spec & plan bloat

Specs grew past 2,500 words; plans piled on another 5,000–10,000 words of code.

2

Shadow code

Plans mandated complete code per step. Agents discarded it and rewrote against the real repo.

3

Inconsistent review

Review folded into a single self-check pass, applied unevenly across tasks.

> The rest of this deck is how chris-code answers each.

The core idea

"Contracts stay, choreography goes."

Contracts stay

Spec invariants

Public APIs & signatures

Data schemas

spec: contracts only

Choreography goes

~~Step-by-step code~~

~~Inline scaffolds~~

~~"do this, then that"~~

plan: what/where, no code

Agents ignored prescriptive code blocks and rewrote them anyway. Plans now say **what** and **where**; code is written against the real repo.

The pipeline

The agent layer

a .py file in a torch project

↓ dispatched by scope (extension + deps) ↓

coder — exclusive

pytorch-coder ✓

~~**python-coder**~~

most-specific wins

quality-reviewer — additive

python-quality-reviewer

pytorch-quality-reviewer

all matching fire

review-lite — commit gate

python-review-lite

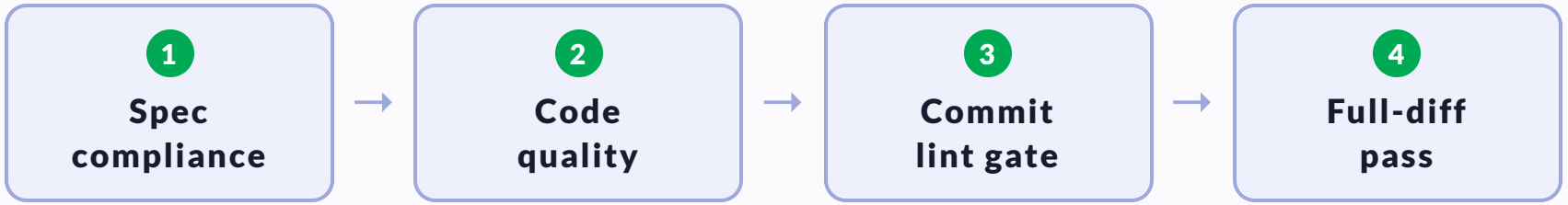
idiom + lint, per commit

design-reviewer — verification gate

python-design-reviewer

senior cohesion, read-only

Review is a uniform, multi-stage gate



The **same** gates on every task. No task is "small enough to skip." **Do Not Trust the Report** — reviewers re-read the code, not the summary. More stages raise *recall*, not proof: they catch more, they don't certify the rest is clean.

Parallelism is a feature, not a footgun

Stage 1 — **Task A · api/** | **Task B · db/** | **Task C · ui/**

file footprints mapped up front · non-overlapping tasks run concurrently

Stage 2 — **Task D · api/ + db/ (waits — overlaps A & B)**

Superpowers flagged parallel implementers as dangerous. chris-code solves it by **scheduling**, not prohibition.

Verification before "done"

No completion claim without fresh evidence. "I'm confident" → run the commands. **Green means these lenses caught nothing, not that nothing's wrong** — the independent axes (deterministic lint, spec-blind intent) carry more than another same-model re-read.

Beyond the pipeline: quality campaigns

On-demand skills for when you want to go hunting:

- `bug-hunt` — parallel adversarial edge-case tests across subsystems
- `test-sweep` — iterative combinatorial test-and-fix campaign
- `code-archaeology` — dead code, stubs, spec-vs-impl gaps
- `technical-review` — math / algorithm / numerical correctness for ML
- `python-review` • `rust-review` — senior refactor & API-design passes
- `release` — version bump + changelog + GitHub release

Remediation & coherent change

A bug, a refactor, an API alignment — the behavior is settled, only the **implementation** is open. One engine handles all of them: find the fix that fits the codebase, and defend it against the alternatives.

remediating-issues · systematic-debugging · lean-spec · direct

↓ feed a **determined** change

coherent-change · research → defend the most coherent fix → implement → lite-review

↓ hand back a working, lite-reviewed change

caller closes · regression / coverage → design-reviewer gate → land

Signature output: a **defended choice** — every candidate rooted in how the codebase already solves it, plus why the chosen one wins. Ship the fix that looks like it was always there.

How to use it

Drop into any repo. Describe a feature, or type `/brainstorming`.

The workflow designs before it codes, keeps the main context clean by offloading to focused subagents, and **catches agent drift before it reaches** `main`.

That's chris-code.